

INFORME TÉCNICO

SISTEMA DE MICROSERVICIOS “NUBEMÉDICA”

Asignatura: Fullstack I

Experiencia de Aprendizaje: Experiencia 3

Tecnologías utilizadas: Spring Boot, JWT, Swagger/OpenAPI, MySQL, WebClient, JUnit5, Mockito.

1. Introducción

El presente informe tiene como objetivo documentar la evolución del proyecto NubeMédica, desarrollado bajo una arquitectura basada en microservicios utilizando el ecosistema Spring Boot.

La solución propuesta corresponde a una plataforma integral de gestión médica orientada a profesionales de la salud, permitiendo administrar agendas médicas, pacientes, fichas clínicas, reportes y sesiones de telemedicina de manera centralizada y segura.

Durante esta experiencia se incorporaron nuevas funcionalidades orientadas a la escalabilidad, seguridad y mantenibilidad del sistema, destacando:

- Servicio de autenticación mediante JWT.
- Integración de Swagger/OpenAPI.
- Comunicación entre microservicios.
- Testing automatizado con JUnit5 y Mockito.
- Persistencia desacoplada mediante el patrón database per service.

El desarrollo fue realizado utilizando control de versiones colaborativo mediante Git y GitHub, aplicando una estrategia de branching para la correcta gestión del código fuente y el trabajo en equipo.

2. Diagramas de Arquitectura y Base de Datos

2.1 Modelo de Arquitectura Actualizado

La arquitectura del sistema fue diseñada siguiendo el paradigma de **Microservicios Descentralizados**, permitiendo desacoplar responsabilidades y facilitar el mantenimiento, despliegue y escalabilidad de la aplicación.

La solución se compone de los siguientes elementos:

API Gateway

Corresponde al punto de entrada único del sistema.
Su función principal es:

- Gestionar el enrutamiento de peticiones.
- Centralizar la seguridad.
- Validar tokens JWT.
- Filtrar solicitudes entrantes.
- Unificar el acceso a los distintos microservicios.

Microservicio de Autenticación (MS-Login)

Encargado de:

- Gestionar credenciales de acceso.
- Generar tokens JWT.
- Validar sesiones activas.
- Administrar procesos de login y logout.

Microservicios de Dominio

La lógica del sistema se divide en múltiples servicios independientes:

- Microservicio de Doctores
- Microservicio de Pacientes
- Microservicio de Calendario
- Microservicio de Ficha Médica
- Microservicio de Telemedicina
- Microservicio de Reportes
- Microservicio de Notificaciones
- Microservicio de Direcciones
- Microservicio de Estados de Cita

Cada servicio posee responsabilidades específicas y opera de forma autónoma.

Persistencia Descentralizada

Se implementó el patrón **Database per Service**, donde cada microservicio administra su propia base de datos.

Este enfoque permite:

- Mayor independencia entre servicios.
- Reducción del acoplamiento.
- Escalabilidad individual.
- Mejor tolerancia a fallos.

Modelo Entidad-Relación

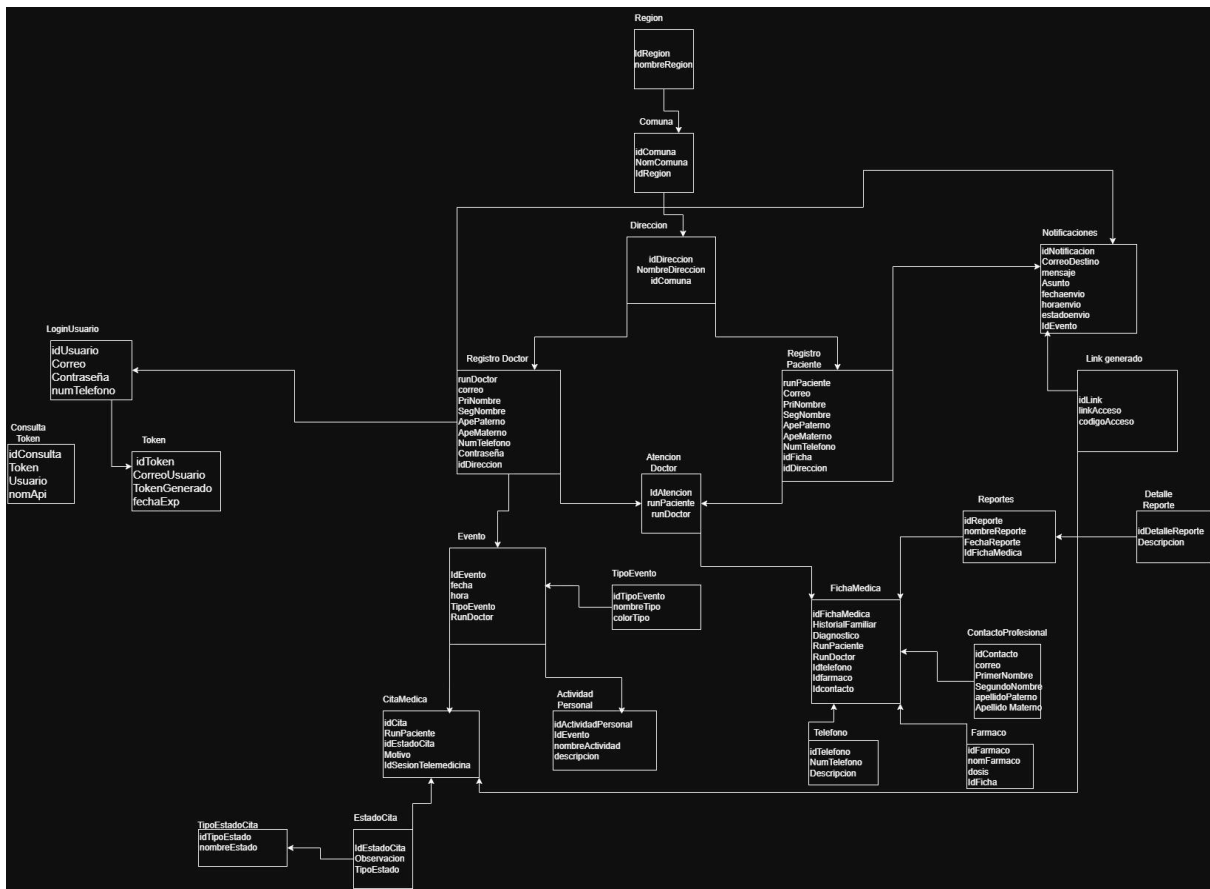
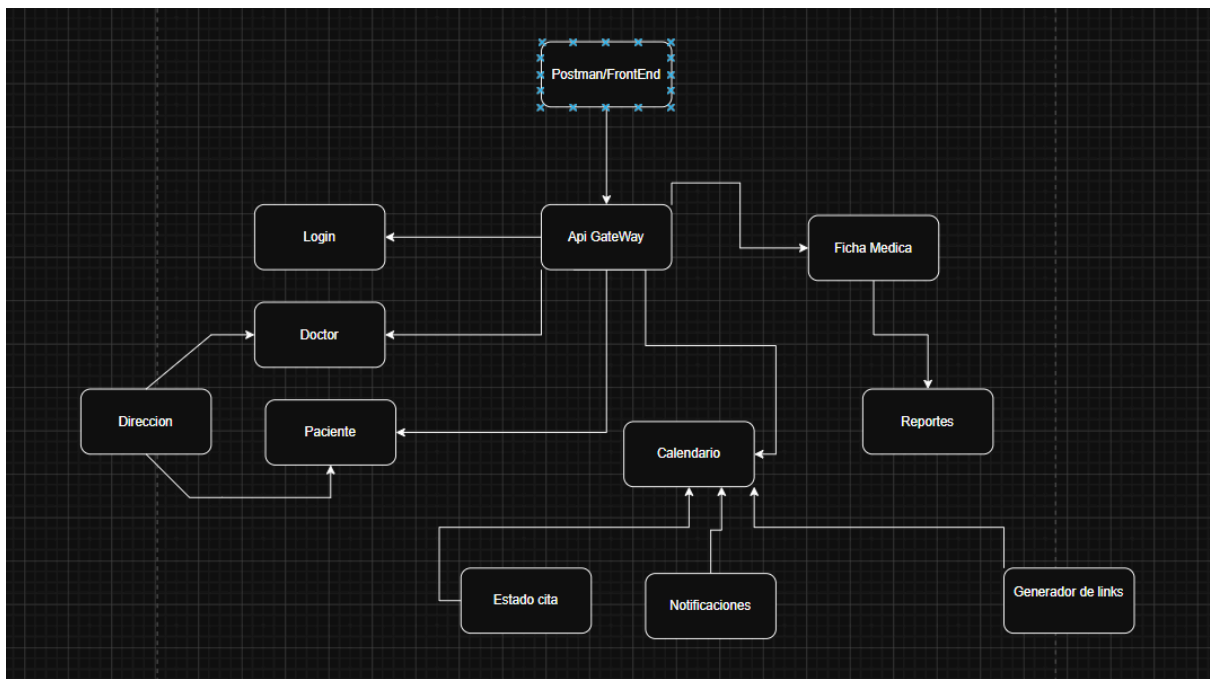


Diagrama de microServicios



3. Servicio de Autenticación (JWT)

3.1 ¿En qué consiste?

El sistema implementa autenticación basada en **JWT (JSON Web Token)**.

JWT permite transmitir información de forma segura entre cliente y servidor mediante objetos JSON firmados digitalmente.

La principal característica de este mecanismo es que funciona de manera **stateless**, es decir, el servidor no necesita almacenar sesiones activas en memoria, ya que toda la información necesaria viaja contenida dentro del token.

Entre los datos almacenados en el token se incluyen:

- RUN del doctor.
- Rol del usuario.
- Fecha de expiración.
- Identificador de sesión.

La firma digital garantiza que el contenido no pueda ser modificado por terceros.

3.2 Implementación en NubeMédica

La autenticación del sistema fue diseñada en dos niveles principales:

Generación y Persistencia del Token (MS-Login)

Cuando un usuario inicia sesión correctamente:

1. El microservicio valida las credenciales.
2. Se genera un JWT firmado utilizando el algoritmo HMAC256.
3. El token es hashado mediante HashUtils.
4. El hash resultante se almacena en la base de datos junto con un indicador active = true.

Este enfoque permite implementar procesos de cierre de sesión efectivos, invalidando tokens previamente emitidos.

Validación y Mutación en API Gateway

El API Gateway incorpora un **AuthenticationFilter** encargado de interceptar todas las solicitudes entrantes.

El proceso consiste en:

1. Validar la firma y expiración del JWT.

2. Extraer los claims del token.
3. Inyectar información del usuario en los headers HTTP.

Los principales headers agregados son:

- `X-Doctor-Run`
- `X-User-Role`

Gracias a esta estrategia, los microservicios internos pueden identificar al usuario autenticado sin necesidad de reprocesar el token JWT en cada solicitud.

4. Implementación por Capas

4.1 Ejemplo: Microservicio de Ficha Médica

Para garantizar escalabilidad, mantenibilidad y separación de responsabilidades, se implementó una arquitectura basada en capas.

Capa DTO

La capa DTO (*Data Transfer Object*) define las estructuras de datos utilizadas para intercambio de información entre cliente y servidor.

Ejemplo:

- `FichaMedicaRequest`
- `FichaMedicaResponse`

Su objetivo es desacoplar las entidades internas de la exposición pública de la API.

Capa Controller

La clase `FichaMedicaController` administra los endpoints REST del microservicio.

Entre sus responsabilidades se encuentran:

- Recepción de solicitudes HTTP.
- Validación de datos mediante `@Valid`.
- Manejo de respuestas HTTP.
- Delegación de lógica hacia la capa de servicio.

Se implementaron operaciones:

- POST

```

@Transactional
public FichaMedicaResponse crearFicha(String runPaciente, String runDoctorToken) {
    validarRelacionDoctorPaciente(runDoctorToken, runPaciente);

    if (fichaMedicaRepository.existsByRunPacienteAndRunDoctor(runPaciente, runDoctorToken)) {
        throw new DatoDuplicadoException(
            "Ya existe una ficha médica para el paciente " + runPaciente +
            " con el doctor " + runDoctorToken
        );
    }

    FichaMedica nuevaFicha = new FichaMedica();
    nuevaFicha.setRunPaciente(runPaciente);
    nuevaFicha.setRunDoctor(runDoctorToken);

    FichaMedica guardada = fichaMedicaRepository.save(nuevaFicha);
    PacienteDTO paciente = obtenerDatosPaciente(runPaciente, token: null);
    return mapearAResponse(guardada, paciente);
}

```

```

private void validarRelacionDoctorPaciente(String runDoctor, String runPaciente) {
    try {
        Boolean existe = webClientBuilder.build()
            .get()
            .uri(urlsMsDoctores + "/existe/{runDoctor}/{runPaciente}", runDoctor, runPaciente)
            .retrieve()
            .bodyToMono(Boolean.class)
            .block();

        if (Boolean.FALSE.equals(existe)) {
            throw new AccesoDenegadoException(
                "Acceso denegado: No tiene relación de atención con el paciente " + runPaciente + "."
            );
        }
    } catch (AccesoDenegadoException e) {
        throw e;
    } catch (Exception e) {
        throw new ComunicacionMicroservicioException(
            "Error de validación en MS-DOCTORES: " + e.getMessage(), e
        );
    }
}

```

POST `http://localhost:8081/api/v1/fichas?runPaciente=22.222.222-2`

Docs Params Authorization Headers 9 Body **Scripts** Settings

Pre-request 1 Use JavaScript to write tests, visualize response, and more.

Post-response

Body Cookies Headers 16 Test Results

{ } JSON Preview Visualize

```

1  {
2    "idFichaMedica": 4,
3    "historialFamiliar": null,
4    "hipotesisDiagnostica": null,
5    "runPaciente": "22.222.222-2",
6    "runDoctor": "12345678-9",
7    "datosPaciente": {
8      "runPaciente": "22.222.222-2",
9      "nombreCompleto": "Carla Andrea Soto Mendoza",
10     "correo": "paciente.ejemplo@gmail.com",
11     "numTelefono": "+56955554444",
12     "direccion": {
13       "idDireccion": 3,
14       "nombre": "Calle Huérfanos 800",
15       "region": "Metropolitana",
16       "comuna": "Maipu"
17     }
18   },
19   "contactos": [],
20   "telefonos": [],
21   "farmacos": [],
22   "reportes": []
23 }

```

- GET - obtener fichas de un doctor

```

// =====
@Transactional(readOnly = true)
public List<FichaMedicaResponse> listarFichasPorDoctor(String runDoctorToken, String token) {
    return fichaMedicaRepository.findByRunDoctor(runDoctorToken).stream()
        .map(ficha -> {
            PacienteDTO paciente = obtenerDatosPaciente(ficha.getRunPaciente(), token);
            return mapearAResponse(ficha, paciente);
        }).collect(Collectors.toList());
}

```

```
Body Cookies Headers 16 Test Results 20
[ {} JSON Preview Visualize
1 [
2   {
3     "idFichaMedica": 1,
4     "historialFamiliar": null,
5     "hipotesisDiagnostica": null,
6     "runPaciente": "4655983-1",
7     "runDoctor": "12345678-9",
8     "datosPaciente": {
9       "runPaciente": "4655983-1",
10      "nombreCompleto": "Harry James Potter Evans",
11      "correo": "harry.potter@hogwarts.cl",
12      "numTelefono": "+569123123123",
13      "direccion": {
14        "idDireccion": 2,
15        "nombre": "Calle 3 diagon",
16        "region": "Metropolitana",
17        "comuna": "Maipu"
18      }
19    },
20    "contactos": [],
21    "telefonos": [],
22    "farmacos": [],
23    "reportes": []
24  },
25  {
26    "idFichaMedica": 4,
27    "historialFamiliar": "Padre con antecedentes de infarto a los 50 años.",
28    "hipotesisDiagnostica": "Hipertensión arterial esencial.",
29    "runPaciente": "22.222.222-2",
30    "runDoctor": "12345678-9",
```



```

"runDoctor": "12345678-9",
"datosPaciente": {
  "runPaciente": "22.222.222-2",
  "nombreCompleto": "Carla Andrea Soto Mendoza",
  "correo": "paciente.ejemplo@gmail.com",
  "numTelefono": "+56955554444",
  "direccion": {
    "idDireccion": 3,
    "nombre": "Calle Huérfanos 800",
    "region": "Metropolitana",
    "comuna": "Maipu"
  }
},
"contactos": [
  {
    "nombres": "Ricardo",
    "apellidos": "Lagos",
    "correo": "r.lagos.cardiologo@clinica.cl"
  }
],
"telefonos": [
  {
    "numTelefono": "+56999887766",
    "descripcion": "Hermana (Patricia)"
  }
],
"farmacos": [
  {
    "nombreFarmaco": "Enalapril",
    "dosis": 10.5
  }
]

```

```

    {
      "nombreFarmaco": "Enalapril",
      "dosis": 10.5
    },
    {
      "nombreFarmaco": "Aspirina",
      "dosis": 100.0
    }
  ],
  "reportes": []
}
]

```

- GET por ID

```
// OBTENER FICHA POR ID
@Transactional(readOnly = true)
public FichaMedicaResponse obtenerFichaPorId(Long idFicha, String runDoctorToken, String token) {
    FichaMedica ficha = fichaMedicaRepository.findById(idFicha)
        .orElseThrow(() -> new RecursoNoEncontradoException(
            "No se encontró la ficha médica con ID: " + idFicha
        ));

    validarRelacionDoctorPaciente(runDoctorToken, ficha.getRunPaciente());

    PacienteDTO paciente = obtenerDatosPaciente(ficha.getRunPaciente(), token);
    return mapearAResponse(ficha, paciente);
}
```

```
{
  "idFichaMedica": 4,
  "historialFamiliar": "Padre con antecedentes de infarto a los 50 años.",
  "hipotesisDiagnostica": "Hipertensión arterial esencial.",
  "runPaciente": "22.222.222-2",
  "runDoctor": "12345678-9",
  "datosPaciente": {
    "runPaciente": "22.222.222-2",
    "nombreCompleto": "Carla Andrea Soto Mendoza",
    "correo": "paciente.ejemplo@gmail.com",
    "numTelefono": "+5695555444",
    "direccion": {
      "idDireccion": 3,
      "nombre": "Calle Huérfanos 800",
      "region": "Metropolitana",
      "comuna": "Maipu"
    }
  },
  "contactos": [
    {
      "nombres": "Ricardo",
      "apellidos": "Lagos",
      "correo": "r.lagos.cardiologo@clinica.cl"
    }
  ],
  "telefonos": [
    {
      "numTelefono": "+56999887766",
      "descripcion": "Hermana (Patricia)"
    }
  ]
}
```

```

    }
  ],
  "farmacos": [
    {
      "nombreFarmaco": "Enalapril",
      "dosis": 10.5
    },
    {
      "nombreFarmaco": "Aspirina",
      "dosis": 100.0
    }
  ],
  "reportes": []
}

```

- PUT

```

// ACTUALIZAR FICHA
@Transactional
public FichaMedicaResponse actualizarFicha(Long idFicha, FichaMedicaUpdateRequest request, String runDoctorToken) {
    FichaMedica ficha = fichaMedicaRepository.findById(idFicha)
        .orElseThrow(() -> new RecursoNoEncontradoException(
            "No se encontró la ficha médica con ID: " + idFicha
        ));

    validarRelacionDoctorPaciente(runDoctorToken, ficha.getRunPaciente());

    ficha.setHistorialFamiliar(request.getHistorialFamiliar());
    ficha.setDiagnostico(request.getHipotesisDiagnostica());

    if (request.getContactosProfesionales() != null) {
        ficha.getContactoPro().clear();
        request.getContactosProfesionales().forEach(dto -> {
            ContactoProfesional cp = new ContactoProfesional();
            cp.setNombres(dto.getNombres());
            cp.setApellidos(dto.getApellidos());
            cp.setCorreo(dto.getCorreo());
            cp.setFichaMedica(ficha);
            ficha.getContactoPro().add(cp);
        });
    }
}

```

```

    if (request.getTelefonosEmergencia() != null) {
        ficha.getTelefonos().clear();
        request.getTelefonosEmergencia().forEach(dto -> {
            TelefonoEmergencia te = new TelefonoEmergencia();
            te.setNumTelefono(dto.getNumTelefono());
            te.setDescripcion(dto.getDescripcion());
            te.setFichaMedica(ficha);
            ficha.getTelefonos().add(te);
        });
    }

    if (request.getFarmacos() != null) {
        ficha.getFarmacos().clear();
        request.getFarmacos().forEach(dto -> {
            FarmacosRecetados fr = new FarmacosRecetados();
            fr.setNombreFarmaco(dto.getNombreFarmaco());
            fr.setDosis(dto.getDosis());
            fr.setFichaMedica(ficha);
            ficha.getFarmacos().add(fr);
        });
    }

    FichaMedica actualizada = fichaMedicaRepository.save(ficha);
    PacienteDTO paciente = obtenerDatosPaciente(actualizada.getRunPaciente(), token);
    return mapearAResponse(actualizada, paciente);
}

```

```

1  {
2      "idFichaMedica": 4,
3      "historialFamiliar": "Padre con antecedentes de infarto a los 50 años.",
4      "hipotesisDiagnostica": "Hipertensión arterial esencial.",
5      "runPaciente": "22.222.222-2",
6      "runDoctor": "12345678-9",
7      "datosPaciente": {
8          "runPaciente": "22.222.222-2",
9          "nombreCompleto": "Carla Andrea Soto Mendoza",
10         "correo": "paciente.ejemplo@gmail.com",
11         "numTelefono": "+56955554444",
12         "direccion": {
13             "idDireccion": 3,
14             "nombre": "Calle Huérfanos 800",
15             "region": "Metropolitana",
16             "comuna": "Maipu"
17         }
18     },
19     "contactos": [
20         {
21             "nombres": "Ricardo",
22             "apellidos": "Lagos",
23             "correo": "r.lagos.cardiologo@clinica.cl"
24         }
25     ],
26     "telefonos": [
27         {
28             "numTelefono": "+56999887766",
29             "descripcion": "Hermana (Patricia)"

```

```

    },
],
"farmacos": [
    {
        "nombreFarmaco": "Enalapril",
        "dosis": 10.5
    },
    {
        "nombreFarmaco": "Aspirina",
        "dosis": 100.0
    }
],
"reportes": []
}

```

- DELETE

```

@Transactional
public void eliminarFicha(Long idFicha, String runDoctorToken) {
    FichaMedica ficha = fichaMedicaRepository.findById(idFicha)
        .orElseThrow(() -> new RecursoNoEncontradoException(
            "No se encontró la ficha médica con ID: " + idFicha
        ));

    validarRelacionDoctorPaciente(runDoctorToken, ficha.getRunPaciente());

    fichaMedicaRepository.deleteById(idFicha);
}

```

```

private void validarRelacionDoctorPaciente(String runDoctor, String runPaciente) {
    try {
        Boolean existe = webClientBuilder.build()
            .get()
            .uri(urlMsDoctores + "/existe/{runDoctor}/{runPaciente}", runDoctor, runPaciente)
            .retrieve()
            .bodyToMono(Boolean.class)
            .block();

        if (Boolean.FALSE.equals(existe)) {
            throw new AccesoDenegadoException(
                "Acceso denegado: No tiene relación de atención con el paciente " + runPaciente + "."
            );
        }
    } catch (AccesoDenegadoException e) {
        throw e;
    } catch (Exception e) {
        throw new ComunicacionMicroservicioException(
            "Error de validación en MS-DOCTORES: " + e.getMessage(), e
        );
    }
}

```



Capa Service

La clase `FichaMedicaService` contiene la lógica de negocio del sistema.

Por ejemplo, al crear una ficha médica:

1. Se valida que el doctor tenga relación activa con el paciente.
2. Se consulta información mediante comunicación interna con el microservicio de Doctores.
3. Se ejecutan reglas de negocio antes de persistir la información.

Esta capa concentra toda la lógica crítica del sistema.

Capa Repository

La persistencia se implementa mediante Spring Data JPA utilizando la interfaz `FichaMedicaRepository`.

Esta capa permite:

- Operaciones CRUD.
- Consultas personalizadas.
- Integración transparente con la base de datos.

4.2 Comunicación entre Microservicios

La comunicación entre servicios se implementa utilizando `WebClient` de Spring WebFlux.

El enfoque utilizado corresponde a comunicación síncrona mediante solicitudes HTTP internas.

Funcionamiento

Cuando un microservicio necesita información externa:

1. Actúa como cliente HTTP.
2. Envía una solicitud a otro servicio dentro de la red privada.
3. Recibe la respuesta.
4. Continúa el flujo de negocio.

Ejemplo de Implementación

El `CalendarioService` requiere generar enlaces de telemedicina para determinadas citas médicas.

Para ello:

1. Envía un `POST` interno al microservicio `MS-Telemedicina`.
2. El servicio genera el enlace correspondiente.
3. La URL es retornada al servicio de calendario.
4. Finalmente se almacena en la cita médica antes de responder al usuario.

5. Swagger (OpenAPI 3)

5.1 ¿Qué es y en qué consiste?

Swagger corresponde a un conjunto de herramientas orientadas a la documentación de APIs REST.

En el proyecto se integró mediante `springdoc-openapi`, permitiendo:

- Documentar endpoints automáticamente.
- Visualizar recursos REST.
- Probar solicitudes desde navegador.
- Facilitar integración frontend-backend.

La documentación se genera dinámicamente a partir de las anotaciones presentes en el código fuente.

5.2 Implementación en Microservicios

Debido a la arquitectura distribuida del sistema, fue necesario implementar una solución centralizada para la documentación.

Configuración Local

Cada microservicio define una clase `OpenApiConfig`, indicando como servidor principal el puerto correspondiente al API Gateway (`localhost:8081`).

Integración con API Gateway

El Gateway implementa un filtro denominado `WebFilterSwagger`.

Su función es:

- Interceptar documentación OpenAPI.
- Inyectar configuración de seguridad `bearerAuth`.
- Permitir autenticación JWT directamente desde Swagger UI.

Gracias a esto, cualquier endpoint protegido puede ser probado desde la interfaz web utilizando tokens válidos.

Pruebas de implementación de swagger en 3 microservicios.

Swagger MS-Doctor

POST CREAR DOCTOR

POST

/api/v1/doctores

Crea un doctor a la base de datos

Utiliza el requestBody para crear un doctor

Parameters

Cancel

Reset

No parameters

Request body required

application/json

```
{  "correo": "doctor.perez@nubemédica",  "runDoctor": "12345678-9",  "prNombre": "jose",  "segNombre": "juan",  "apaApellido": "perez",  "apaApellido": "poblete",  "telefono": "+569123123",  "contrasena": "Admin12345",  "direccion": {    "nombre": "Glorias navales 790",    "comuna": "Maipu",    "region": "Metropolitana"  }}
```

Execute

Clear

Code

Details

201

Undocumented

Response body

```
{  "runDoctor": "12345678-9",  "correo": "doctor.perez@nubemédica",  "nombreCompleto": "jose juan perez poblete",  "telefono": "+569123123",  "direccion": {    "idDireccion": 1,    "nombre": "Glorias navales 790",    "region": "Metropolitana",    "comuna": "Maipu"  }}
```

Download

PUT DOCTOR

PUT

/api/v1/doctores/{runDoctor}

Actualizar informacion de un doctor utilizando su rut

Parameters

Cancel

Reset

Name	Description
runDoctor * required	
string	
(path)	12345678-9

Request body required

application/json

```
{  "prNombre": "juan",  "segNombre": "jose",  "apaApellido": "perez",  "apaApellido": "poblete",  "telefono": "+569123123",  "correo": "doctor.perez@nubemédica.cl",  "direccion": {    "nombre": "Glorias navales 790",    "comuna": "Maipu",    "region": "Metropolitana"  }}
```

200

Response body

```
{  "runDoctor": "12345678-9",  "correo": "doctor.perez@nubemédica.cl",  "nombreCompleto": "juan jose perez poblete",  "telefono": "+569123123",  "direccion": {    "idDireccion": 1,    "nombre": "Glorias navales 790",    "region": "Metropolitana",    "comuna": "Maipu"  }}
```

Download

GET Obtener datos propios del doctor

GET

/api/v1/doctores/{runDoctor} Se obtiene informacion del doctor por RUT

Solo puede obtener informacion de si mismo.

Parameters

Cancel

Name	Description
runDoctor * required	
string	
(path)	

Execute

Clear

200

Response body

```
{
  "runDoctor": "12345678-9",
  "correo": "doctor.perez@nubemédica.cl",
  "nombreCompleto": "juan jose perez poblete",
  "telefono": "+569123123",
  "direccion": {
    "idDireccion": 1,
    "nombre": "Glorias navales 790",
    "region": "Metropolitana",
    "comuna": "Maipú"
  }
}
```

Download

GET ATENCION por rut de paciente

GET

/api/v1/atenciones/paciente/{runPaciente} Lista el paciente en especifico del doctor por el rut del paciente

Parameters

Try it out

Name	Description
runPaciente * required	
string	
(path)	

Responses

200

Response body

```
[
  {
    "id": 1,
    "doctor": {
      "runDoctor": "12345678-9",
      "prNombre": "juan",
      "segNombre": "jose",
      "apaPaterno": "perez",
      "apaMaterno": "poblete",
      "telefono": "+569123123",
      "correo": "doctor.perez@nubemédica.cl",
      "idDireccion": 1,
      "datosDireccion": null
    },
    "datosPaciente": null
  }
]
```

Download

GET verificar asociacion doctor-paciente

GET

/api/v1/atenciones/existe/{runDoctor}/{runPaciente}

Verifica si existe una relacion entre doctor y un paciente

se necesita el rut paciente y doctor

Parameters

Cancel

Name	Description
runDoctor * required string (path)	12345678-9
runPaciente * required string (path)	4655983-1

Execute

Clear

Code

Details

200

Response body

```
true
```

Download

GET obtener todos los pacientes asociados al doctor por atenciones

GET

/api/v1/atenciones/doctor

Lista todos los pacientes pertenecientes a un doctor por su RUT

Parameters

Cancel

Name	Description
X-Doctor-Run * required string (header)	12345678-9

Execute

Clear

Code

Details

200

Response body

```
{
  "id": 1,
  "doctor": {
    "runDoctor": "12345678-9",
    "prNombre": "Juan",
    "segNombre": "Jose",
    "apaPaterno": "perez",
    "apaMaterno": "poblete",
    "telefono": "9569123123",
    "correo": "doctor.perez@nubemedita.cl",
    "idDireccion": 1,
    "datosDireccion": {
      "idDireccion": 4,
      "nombre": "Glorias navales 790",
      "region": "Metropolitana",
      "comuna": "Maipo"
    }
  },
  "datosPaciente": {
    "runPaciente": "4655983-1",
    "nombres": "Harry James",
    "apellidos": "Potter Evans"
  }
}
```

Download

GET obtener el rut del paciente

GET

/api/v1/atenciones/doctor/{runDoctor}

Lista solo los rut de los pacientes pertenecientes a un doctor por su Rut

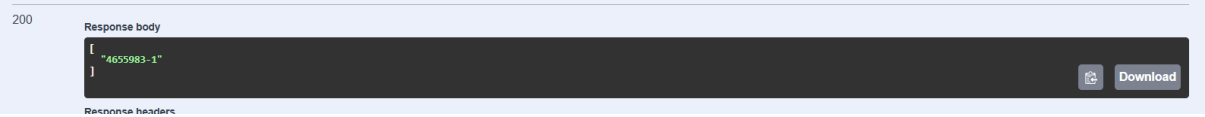
Parameters

Cancel

Name	Description
runDoctor * required string (path)	12345678-9

Execute

Clear



DELETE asociación paciente doctor

DELETE `/api/v1/atenciones/{runPaciente}/desasociar` Elimina la relacion entre paciente y doctor

Requiere el rut del doctor y paciente

Parameters

Cancel

Reset

Name	Description
runPaciente * required	
string (path)	4655983-1

Request body required

application/json

```
"1235678-9"
```



Swagger MS-Paciente

Post Crear paciente

POST `/api/v1/pacientes` Crear un paciente

Requiere datos del paciente y rut del doctor

Parameters

Cancel

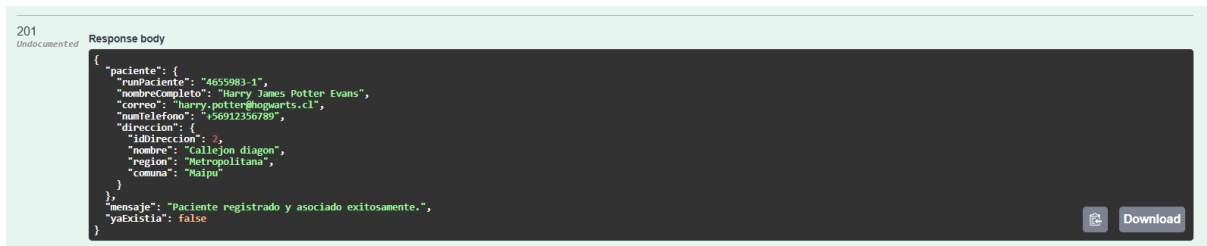
Reset

Name	Description
X-Doctor-Run * required	
string (header)	12345678-9

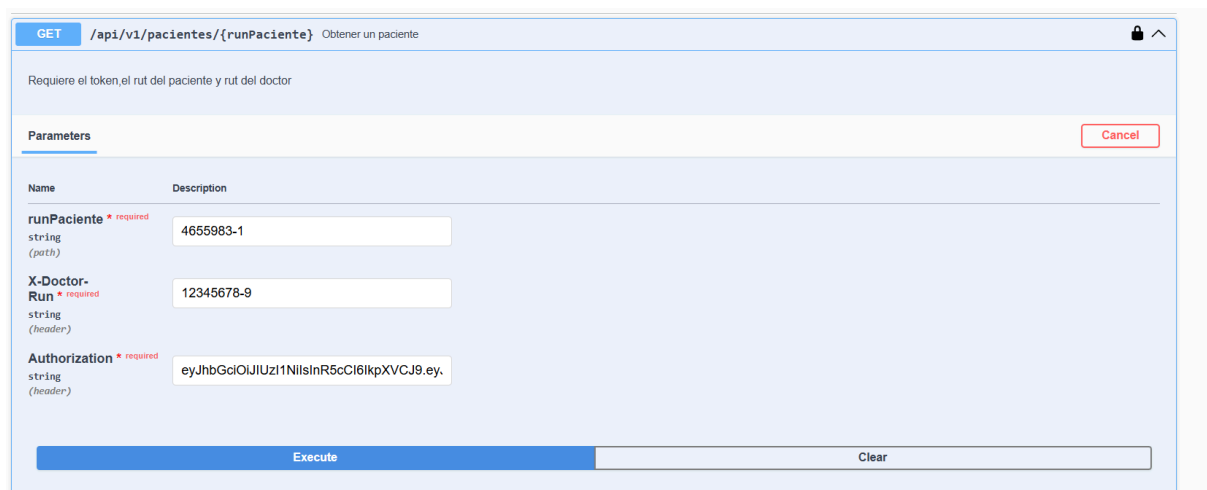
Request body required

application/json

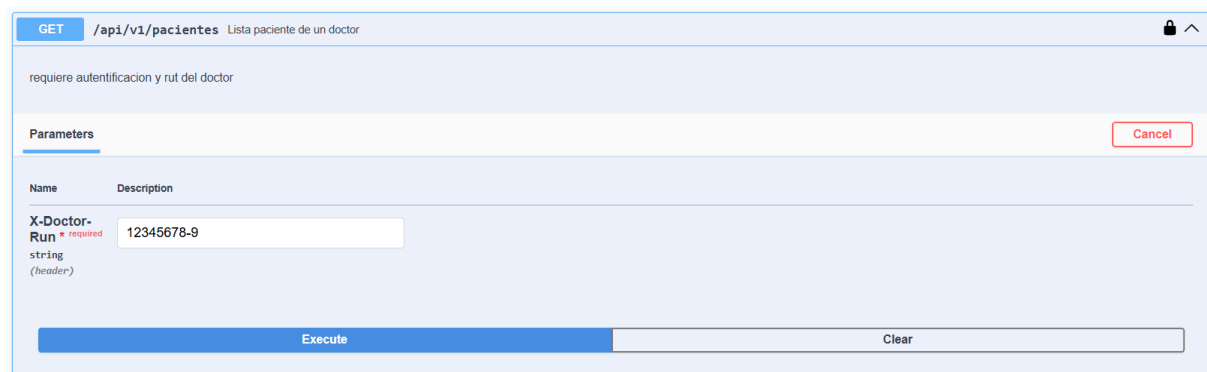
```
{
  "runPaciente": "4655983-1",
  "correo": "harry.potter@hogwarts.cl",
  "prNombre": "Harry",
  "segNombre": "James",
  "apPaterno": "Potter",
  "apMaterno": "Evans",
  "nroTelefono": "+56912356789",
  "direccion": {
    "nombre": "callejon diagon",
    "comuna": "Saipe",
    "region": "Metropolitana"
  }
}
```

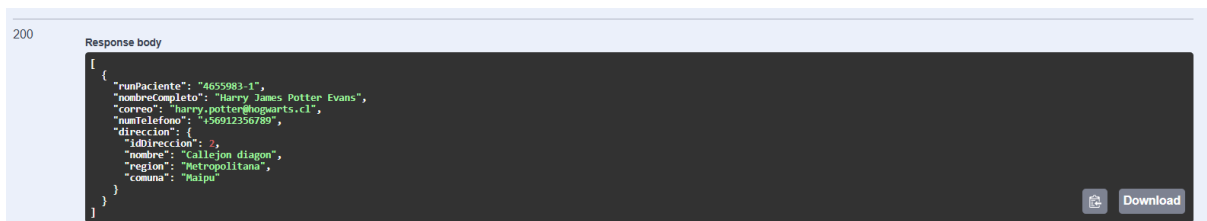


GET paciente de un doctor

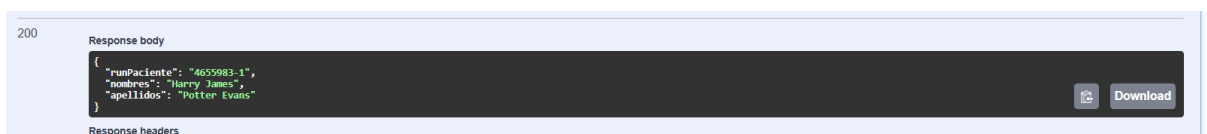
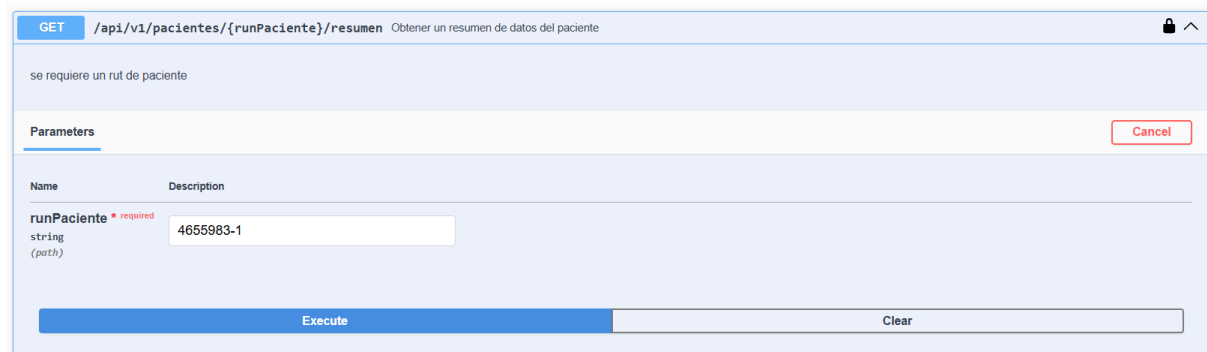


GET de todos los paciente pertenecientes a un doctor

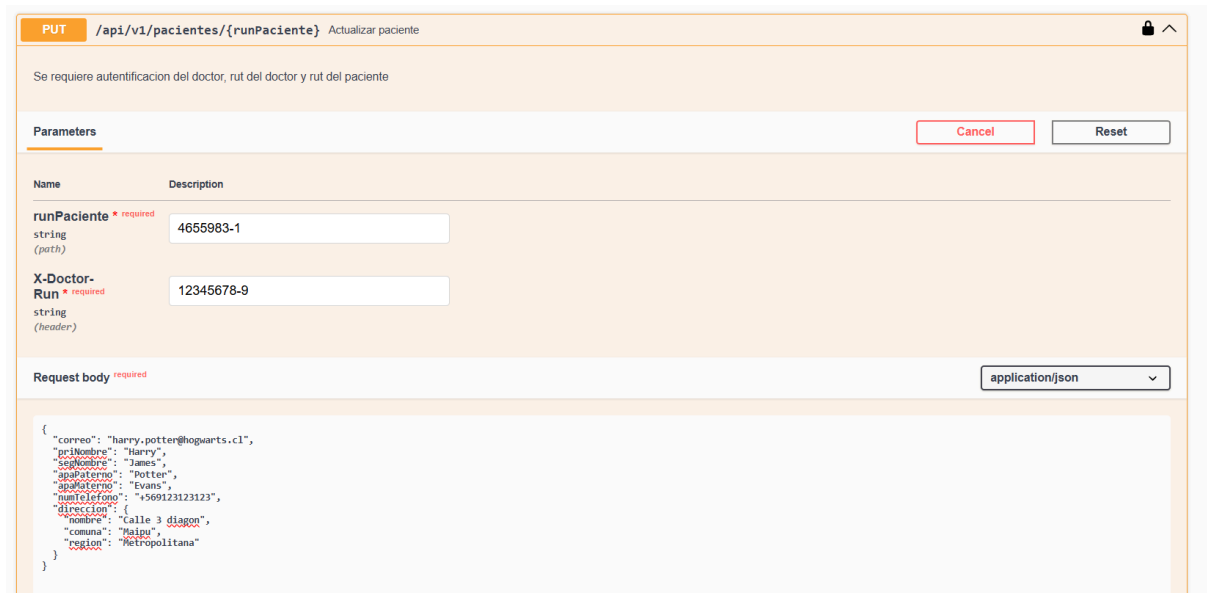


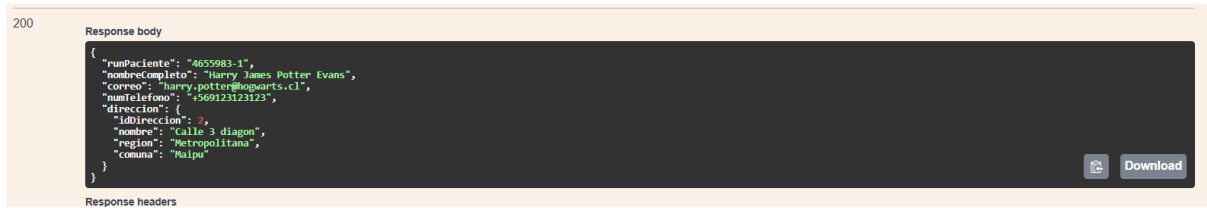


GET resumen de paciente



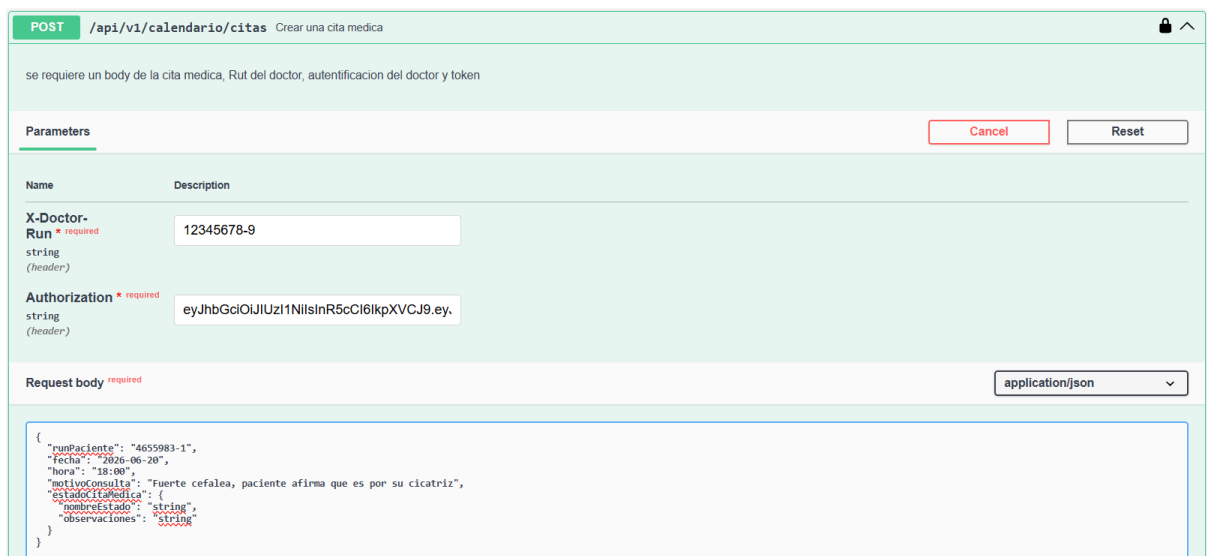
PUT modificar paciente





Swagger MS-Calendarario

POST Cita medica



GET Obtener cita medica por su ID

GET

/api/v1/calendario/citas/{id}

Obtener cita medica por ID

Se requiere ID de cita, RUT del doctor, autentificacion

Parameters

Cancel

Name	Description
id * required integer(\$int64) (path)	1
X-Doctor-Run * required string (header)	12345678-9

Execute

Clear

200

Response body

```
{
  "idEvento": 1,
  "runPaciente": "4655983-1",
  "runDoctor": "12345678-9",
  "fecha": "2026-06-20",
  "hora": "18:00:00",
  "motivoConsulta": "Fuerte cefalea, paciente afirma que es por su cicatriz",
  "estadoCita": {
    "nombreEstado": "Agendada",
    "observaciones": "Cita creada desde Calendario"
  },
  "telemedicina": {
    "idSesionTelemedicina": 1,
    "linkAcceso": "https://nube-medica.com/teleconsulta/85e90597",
    "codigoAcceso": "39K3R2"
  }
}
```

Download

PUT modificar citamedica

PUT

/api/v1/calendario/citas/{id}

Actualizar cita medica por su ID

Se requiere ID de la cita, body de la cita, RUT del doctor y token del doctor

Parameters

Cancel

Reset

Name	Description
id * required integer(\$int64) (path)	1
X-Doctor-Run * required string (header)	12345678-9
Authorization * required string (header)	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ

Request body

required

application/json

```
{
  "runPaciente": "4655983-1",
  "fecha": "2026-06-20",
  "hora": "19:00",
  "motivoConsulta": "Fuerte cefalea, paciente afirma que es por su cicatriz",
  "estadoCitaMedica": {
    "nombreEstado": "Cancelada",
    "observaciones": "Paciente dice que fue momentaneo."
  }
}
```

200

Response body

```
{
  "idEvento": 1,
  "runPaciente": "4655983-1",
  "runDoctor": "12345678-9",
  "fecha": "2026-06-20",
  "hora": "19:00:00",
  "motivoConsulta": "Fuerte cefalea, paciente afirma que es por su cicatriz",
  "estadoCita": {
    "nombreEstado": "Cancelada",
    "observaciones": "Paciente dice que fue momentaneo."
  },
  "telemedicina": {
    "idSesionTelemedicina": 1,
    "linkAcceso": "https://nube-medica.com/teleconsulta/85e90597",
    "codigoAcceso": "39K3R2"
  }
}
```

Download

POST Actividad personal

POST

/api/v1/calendario/actividades

Crear actividades personales de un doctor

se requiere body de actividades personal, RUT doctor, Token

Parameters

Cancel

Reset

Name	Description
X-Doctor-Run * required	12345678-9
string (header)	
Authorization * required	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ
string (header)	

Request body required

application/json

```
{  "fecha": "2026-06-20",  "hora": "20:00",  "nombreActividad": "Ir al mundial de quidditch",  "descripcion": "voy a ir a la barra brava Irlanda"}
```

201

Undocumented

Response body

```
{  "idEvento": 2,  "fecha": "2026-06-20",  "hora": "20:00:00",  "nombreActividad": "Ir al mundial de quidditch",  "descripcion": "voy a ir a la barra brava Irlanda"}
```

Download

GET buscar actividad personal por id de evento

GET

/api/v1/calendario/actividades/{id}

Obtener actividades personales por ID

se requiere ID de actividad,RUT doctor, TOKEN doctor

Parameters

Cancel

Name	Description
id * required	2
integer(\$int64) (path)	
X-Doctor-Run * required	12345678-9
string (header)	

Execute

Clear

200

Response body

```
{  "idEvento": 2,  "fecha": "2026-06-20",  "hora": "20:00:00",  "nombreActividad": "Ir al mundial de quidditch",  "descripcion": "voy a ir a la barra brava Irlanda"}
```

Download

Response headers

PUT de actividad personal

PUT /api/v1/calendario/actividades/{id} Actualizar actividad personal

se requiere ID de actividad personales, BODY actividad personal, RUT doctor y autentificación

Parameters Cancel Reset

Name	Description
id * required integer(\$int64) (path)	2
X-Doctor-Run * required string (header)	12345678-9
Authorization * required string (header)	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ

Request body * required application/json

```
{
  "fecha": "2026-06-20",
  "hora": "21:00",
  "nombreActividad": "Ir al mundial de quidditch",
  "descripcion": "era otra hora"
}
```

200

Response body

```
{
  "idEvento": 1,
  "fecha": "2026-06-20",
  "hora": "21:00:00",
  "nombreActividad": "Ir al mundial de quidditch",
  "descripcion": "era otra hora"
}
```

Download

Response headers

GET obtener todas las citas del doctor

GET /api/v1/calendario/citas/mias Obtener todas las citas medicas del doctor

Se requiere token del doctor y RUT del doctor

Parameters Cancel

Name	Description
X-Doctor-Run * required string (header)	12345678-9

Execute Clear

200

Response body

```
{
  "idEvento": 1,
  "rutPaciente": "4655983-1",
  "rutDoctor": "12345678-9",
  "fecha": "2026-06-20",
  "hora": "19:00:00",
  "motivoConsulta": "Fuerte cefalea, paciente afirma que es por su cicatriz",
  "estadoCita": {
    "nombreEstado": "Cancelada",
    "observaciones": "Paciente dice que fue momentaneo."
  },
  "telomedicina": {
    "idSesionTelomedicina": 1,
    "linkAcceso": "https://nube-medica.com/teleconsulta/85e90597",
    "codigoAcceso": "39K3R2"
  }
}
```

Download

GET obtener todas los eventos del doctor

GET

/api/v1/calendario/agenda/doctor

Obtener agenda completa de un doctor

Parameters

Cancel

Name	Description
X-Doctor-Run * required	12345678-9
string	
(header)	

Execute

Clear

200

Response body

```
[
  {
    "idEvento": 1,
    "fecha": "2026-06-20",
    "hora": "19:00:00",
    "tipo": "Cita M\u00e9dica",
    "color": "#E33333",
    "runDoctor": "12345678-9",
    "runPaciente": "4655983-1",
    "motivoConsulta": "Fuerte cefalea, paciente afirma que es por su cicatriz",
    "estadoCitaMedica": {
      "nombreEstado": "Cancelada",
      "observaciones": "Paciente dice que fue momentaneo."
    }
  },
  {
    "idEvento": 2,
    "fecha": "2026-06-20",
    "hora": "21:00:00",
    "tipo": "Actividad Personal",
    "color": "#188853",
    "runDoctor": "12345678-9",
    "nombreActividad": "Ir al mundial de quidditch",
    "descripcion": "era otra hora"
  }
]
```

Download

GET obtener todas las actividades personales

GET

/api/v1/calendario/actividades/mias

Obtener todas las actividades personales de un doctor

Se requiere RUT del doctor y su token

Parameters

Cancel

Name	Description
X-Doctor-Run * required	12345678-9
string	
(header)	

Execute

Clear

200

Response body

```
[
  {
    "idEvento": 2,
    "fecha": "2026-06-20",
    "hora": "21:00:00",
    "nombreActividad": "Ir al mundial de quidditch",
    "descripcion": "era otra hora"
  }
]
```

Download

Response headers

DELETE cita medica

DELETE

/api/v1/calendario/citas/{id}

Eliminar una cita por ID



se requiere ID de la cita, RUT del doctor, y token del doctor

Parameters

Cancel

Name	Description
id * required integer(\$int64) (path)	1
X-Doctor-Run * required string (header)	12345678-9
Authorization * required string (header)	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

Execute

Clear

Code

204

Undocumented

DELETE actividad personal

DELETE

/api/v1/calendario/actividades/{id}

Eliminar actividad personal



se requiere ID actividad personal, RUT doctor y token

Parameters

Cancel

Name	Description
id * required integer(\$int64) (path)	2
X-Doctor-Run * required string (header)	12345678-9

Execute

Clear

DELETE todos los eventos doctor

DELETE

/api/v1/calendario/agenda/doctor/{runDoctor} Limpiar la agenda de un doctor

🔒 ⬆

Parameters

Cancel

Name	Description
runDoctor * required string (path)	1235678-9

Execute

Clear

Code

204

Undocumented

6. Testing y Calidad de Software

6.1. ¿En qué consiste y por qué es importante?

Dentro de una arquitectura basada en microservicios, el proceso de testing adquiere un rol fundamental para garantizar la estabilidad, confiabilidad y mantenibilidad del sistema.

En el proyecto **NubeMédica**, las pruebas fueron diseñadas para validar el comportamiento individual de cada componente de manera aislada, permitiendo verificar tanto la lógica de negocio como la correcta comunicación entre servicios.

La implementación de testing automatizado aporta múltiples beneficios al sistema:

Validación de la lógica de negocio

Las pruebas permiten asegurar que las reglas críticas del sistema se ejecuten correctamente. Entre ellas destacan:

- Validación de traslape de horarios médicos.
- Control de relaciones doctor–paciente.
- Restricciones de acceso a fichas médicas.
- Gestión de estados de citas.
- Persistencia de reportes clínicos.

Esto garantiza que el sistema mantenga consistencia funcional incluso ante escenarios complejos.

Desarrollo desacoplado

Gracias al uso de *Mocks* mediante Mockito, es posible probar cada microservicio sin necesidad de levantar el ecosistema completo.

Esto permite:

- Reducir dependencias entre servicios.
 - Ejecutar pruebas de forma rápida.
 - Simular respuestas externas.
 - Detectar errores de manera temprana.
-

Seguridad durante la refactorización

El testing automatizado proporciona una red de seguridad frente a modificaciones futuras del código.

De esta forma, cualquier cambio estructural o mejora interna puede validarse automáticamente, asegurando que funcionalidades previamente implementadas continúen operando correctamente y evitando regresiones.

6.2. Metodología de Pruebas Unitarias (AAA)

Todas las pruebas del proyecto fueron desarrolladas utilizando el patrón **AAA (Arrange, Act, Assert)**, una metodología ampliamente utilizada para estructurar pruebas unitarias de forma clara y mantenible.

Arrange (Organizar)

En esta etapa se configura el entorno necesario para ejecutar la prueba.

Dentro de NubeMédica se utilizaron:

- Objetos Mock mediante Mockito.
- Simulación de repositorios.
- Simulación de llamadas HTTP con WebClient.
- Configuración manual de propiedades utilizando `ReflectionTestUtils`.

Un aspecto técnico importante fue la inyección manual de URLs de microservicios utilizadas normalmente desde `application.properties`, permitiendo aislar completamente las pruebas del entorno real.

Act (Actuar)

En esta fase se ejecuta directamente el método que se desea validar.

Algunos ejemplos incluyen:

- `crearCitaMedica()`
- `guardarPaciente()`
- `registrarDoctor()`
- `actualizarFicha()`

El objetivo es evaluar el comportamiento real del servicio bajo diferentes escenarios.

Assert (Afirmar)

Finalmente, se verifican los resultados obtenidos utilizando métodos de validación como:

- `assertEquals()`
- `assertNotNull()`
- `assertThrows()`
- `verify()`

Además de validar resultados, también se comprobó la interacción correcta entre servicios, repositorios y dependencias simuladas.

6.3. Implementación de Testing en NubeMédica

El proyecto alcanzó una cobertura aproximada del 60% de la lógica crítica del backend mediante pruebas desarrolladas con **JUnit 5** y **Mockito**.

Las pruebas se enfocaron principalmente en la capa de servicios, debido a que allí reside la mayor complejidad funcional del sistema.

A. Pruebas de Lógica de Negocio

Microservicios: MS-Calendario y MS-Reportes

MS-Calendario

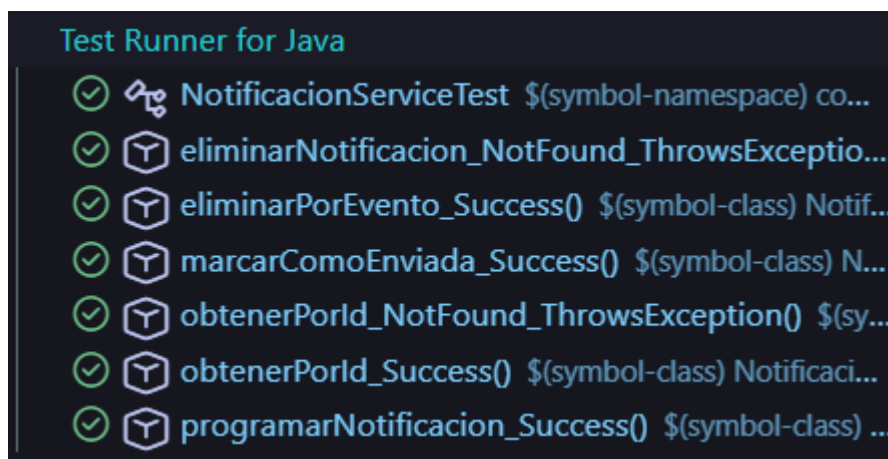
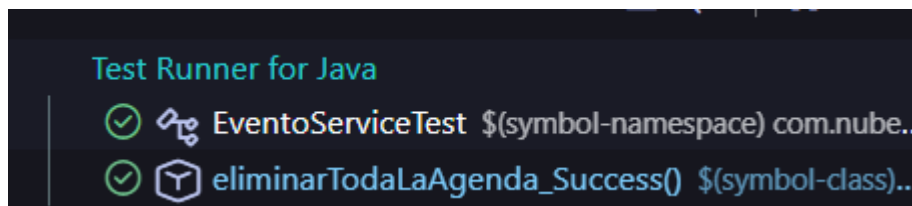
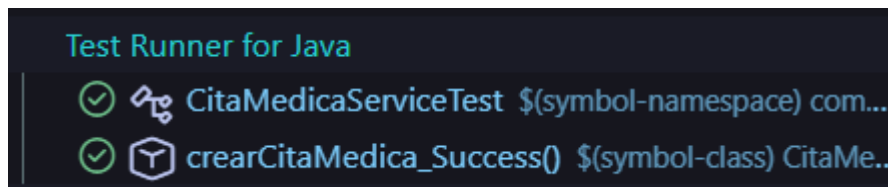
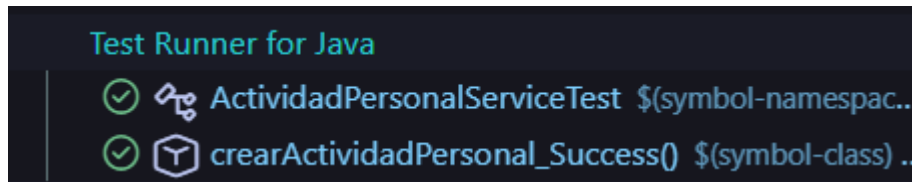
En `ActividadPersonalServiceTest` se validó que una actividad médica no pueda registrarse con fechas anteriores a la actual.

Por otro lado, en `CitaMedicaServiceTest` se simuló exitosamente la orquestación de múltiples microservicios externos.

Durante la prueba se mockearon respuestas provenientes de:

- MS-Notificaciones

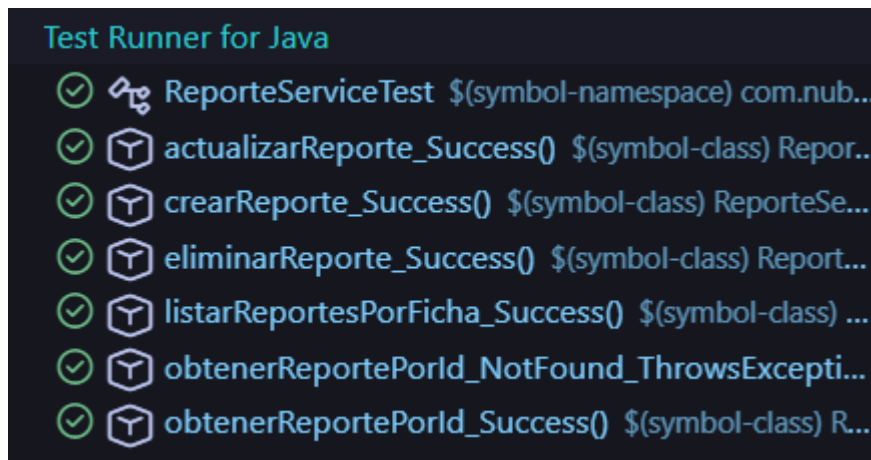
El objetivo fue asegurar que una cita médica solo pueda persistirse cuando todos los servicios externos responden correctamente.



MS-Reportes

En este microservicio se evaluó la persistencia en cascada de entidades clínicas.

Las pruebas verificaron que, al guardar un reporte médico, su correspondiente `DetalleReporte` también sea almacenado correctamente en base de datos, manteniendo la integridad relacional.



B. Pruebas de Resiliencia y Compensación

Microservicio: MS-Paciente

Uno de los escenarios más relevantes implementados corresponde a la prueba:

`asociarYCrearFicha_FallaFicha_RevierteAsociacion`

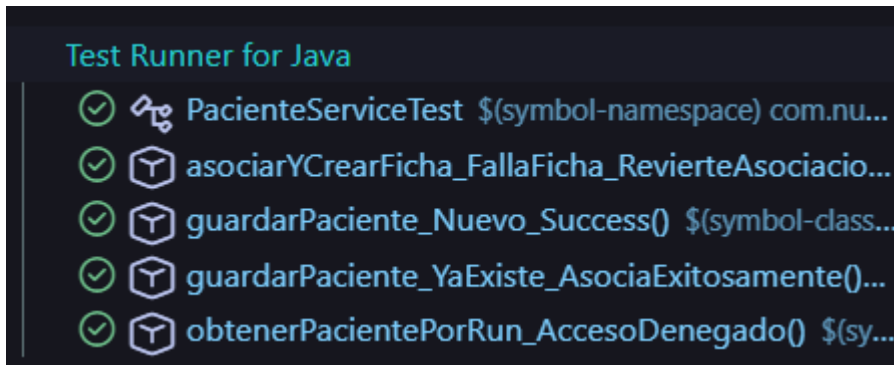
Esta prueba valida una lógica de compensación distribuida entre microservicios.

El flujo probado fue el siguiente:

1. Se asocia correctamente un doctor a un paciente.
2. Se intenta crear la ficha médica.
3. La creación de ficha falla debido a un error externo.

Frente a este escenario, el sistema ejecuta automáticamente una operación compensatoria, realizando una solicitud `DELETE` al microservicio de doctores para revertir la asociación inicial y evitar inconsistencias de datos.

Este tipo de validaciones resulta especialmente importante en arquitecturas distribuidas.



C. Pruebas de Seguridad y Acceso

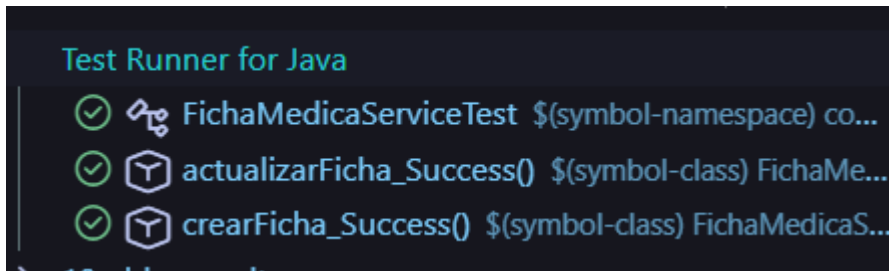
Microservicios: MS-FichaMedica y MS-Doctor

MS-FichaMedica

En la prueba **actualizarFicha_Success** se verificó la correcta actualización de información clínica, incluyendo:

- Diagnósticos.
- Tratamientos.
- Fármacos.
- Observaciones médicas.

Además, se validó que las listas internas de datos clínicos no quedaran vacías tras la actualización.

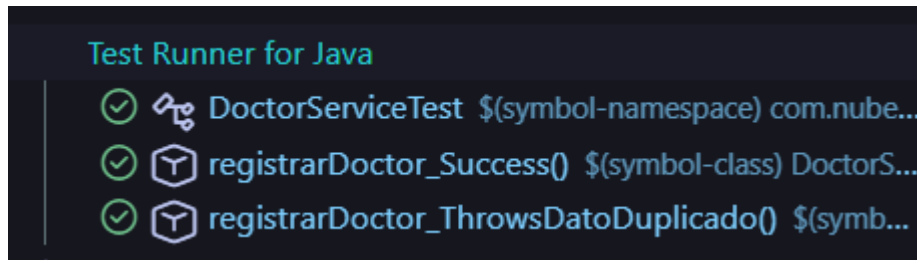


MS-Doctor

En la prueba **registrarDoctor_ThrowsDatoDuplicado** se comprobó que el sistema lance excepciones personalizadas cuando se intenta registrar:

- Un RUN ya existente.
- Un correo electrónico previamente utilizado.

Esto garantiza el cumplimiento de restricciones de integridad y unicidad dentro del sistema.



D. Pruebas de Integración con WebClient

Debido a la naturaleza distribuida del proyecto, se implementaron métodos auxiliares (`mockGet`, `mockPost`) para simular el comportamiento completo de `WebClient`.

Esto permitió reproducir cadenas de llamadas como:

- `.uri()`
- `.header()`
- `.retrieve()`
- `.bodyToMono()`

Gracias a esta estrategia fue posible validar cómo reaccionan los microservicios frente a:

- Respuestas exitosas.
- Errores HTTP 404.
- Errores HTTP 500.
- Fallas de comunicación externas.

6.4. Documentación de Cobertura

La estrategia de testing se enfocó principalmente en la **Capa de Servicio (Service Layer)**, debido a que allí se concentra la lógica crítica y la orquestación de procesos del sistema.

Para lograr un entorno de pruebas desacoplado y controlado, se mockearon:

- Repositorios JPA.
- Servicios externos.
- Clientes WebClient.
- Dependencias internas.

Como resultado, se obtuvo una cobertura cercana al 60% del backend, permitiendo validar de forma confiable:

- Flujos de negocio.
- Reglas de validación.

- Seguridad de acceso.
- Comunicación entre microservicios.
- Manejo de errores y resiliencia.

La implementación de estas pruebas proporciona una base sólida para futuras mejoras y garantiza la estabilidad operativa de la plataforma NubeMédica.